

09 — Tries

DSA in Python — Interview Prep

Tries — Tutorial

Covers the 10 problems solved in this folder. A trie's core idea is always the same: turn each item (a word, a number's bits, a combined key) into a **path** through a tree, one symbol per edge, so that items sharing a common prefix automatically share the same nodes for that prefix. Every problem here is really "what do I store at/along that path, and what do I do with the path once it's built" — the tree structure itself barely changes.

The patterns

1. The foundational structure (01)

A `TrieNode` is a dict of `character -> child node` plus a flag for "a word ends here." `insert` walks/creates the path for a word; `search` walks it and checks the end flag; `starts_with` walks it and checks only that the path exists. Every other problem in this folder is this same skeleton with something added: a different thing stored at each node, a different traversal rule, or both.

2. Wildcard search via DFS instead of a single path walk (02)

Once a query can contain a wildcard, "walk the one matching path" stops being enough — a `.` means "branch into every child here." `add_and_search_word`'s `search` becomes a DFS: normal characters narrow to one child as before, but `.` recursively tries every child and succeeds if **any** branch leads to a match.

3. Stopping at the first match instead of the full path (03)

`replace_words` walks a word down the dictionary trie and returns as soon as it hits an `is_end` node — not because the word ran out, but because a **shorter** match was found and nothing shorter could exist (anything found earlier in the walk is, by construction, shorter than anything found later).

4. Constrained traversal — descend only where a rule allows (04)

`longest_word_in_dictionary` DFS's the trie, but only continues into a child if that child is **itself** a complete word (`is_end`). This encodes "every prefix must also be in the list" directly as a traversal rule instead of checking all prefixes of each candidate word separately.

5. Storing an aggregate at every node, not just at the end (05)

`map_sum_pairs` stores a running sum at **every** node along each inserted key's path (not just the terminal node), so `sum(prefix)` is a single $O(\text{len}(\text{prefix}))$ walk to the prefix's node and a direct read — no subtree scan needed at query time. The delta-based update handles overwriting an existing key's value correctly.

6. Trie-guided backtracking over a grid (06)

`word_search_ii` combines a trie with the grid-DFS backtracking style from other topics: walk the grid and the trie **together**, only stepping to a grid neighbor if that letter exists as a child of the current trie node. This prunes paths no word could ever complete, avoiding a separate independent search per word (which would re-walk shared prefixes of the grid repeatedly).

7. Precomputing bounded results at every node (07)

`search_suggestions_system` is the "autocomplete" trie: insert products in sorted order, capping each node's stored suggestion list at 3 entries so only the lexicographically smallest ones accumulate there. This precomputes every possible query's answer *during insertion*, making each query just a walk-and-read, no sorting or filtering at query time.

8. Reversing the string to flip suffix into prefix (08)

Trie structure only exposes prefix relationships naturally. `short_encoding_of_words` needs *suffix* relationships ("is this word a suffix of another"), so it inserts every word *reversed* — suffix of the original becomes prefix of the reversed string, which the trie handles for free. A word only needs its own encoding entry if its reversed-path node is a leaf (nothing extends past it, so it isn't a suffix of anything longer already covered).

9. Trie over bits instead of characters (09)

`maximum_xor_of_two_numbers` is the biggest conceptual jump in the folder: the trie's alphabet is just `{0, 1}`, one level per bit of a fixed-width number. This makes tries applicable well beyond strings — anything with a fixed-length symbol sequence (bits, digits, tokens) can be trie-indexed the same way. Greedily walking toward the *opposite* bit at each level (to maximize XOR) is a bit-level version of pattern 4's "traversal rule enforces a constraint" idea.

10. Combined keys to answer a two-part query (10)

A plain trie answers one-dimensional questions ("starts with X"). `prefix_and_suffix_search` needs "starts with X *and* ends with Y" simultaneously — solved by inserting a synthetic combined key (`suffix + "#" + word`) for every possible suffix length of every word, turning a query into an ordinary single-path trie lookup of `suffix + "#" + prefix`. This "encode two constraints into one key" trick is worth remembering as a general escape hatch whenever a trie needs to answer more than a single prefix condition at once.

Complexity summary

#	Problem	Time	Extra space
01	<code>implement_trie</code>	$O(\text{len})$ per op	$O(\text{total chars inserted})$
02	<code>add_and_search_word</code>	$O(\text{len})$ exact, $O(26^{\text{len}})$ worst case with wildcards	$O(\text{total chars})$
03	<code>replace_words</code>	$O(\text{total sentence length})$	$O(\text{total dictionary chars})$
04	<code>longest_word_in_dictionary</code>	$O(\text{total chars})$	$O(\text{total chars})$
05	<code>map_sum_pairs</code>	$O(\text{len}(\text{key}))$ per op	$O(\text{total chars})$
06	<code>word_search_ii</code>	$O(\text{rows} \cdot \text{cols} \cdot 4^L)$ worst case ($L = \text{max word length}$)	$O(\text{total dictionary chars})$
07	<code>search_suggestions_system</code>	$O(\text{total chars} \cdot \log(\text{total}) \text{ for sort})$	$O(\text{total chars})$
08	<code>short_encoding_of_words</code>	$O(\text{total chars})$	$O(\text{total chars})$
09	<code>maximum_xor_of_two_numbers</code>	$O(n \cdot \text{bit_length})$	$O(n \cdot \text{bit_length})$
10	<code>prefix_and_suffix_search</code>	$O(\text{len}(\text{word})^2)$ per insert (all suffixes), $O(\text{len})$ per query	$O(\text{total chars}^2)$

10's space cost is the notable outlier — inserting every suffix of every word is quadratic in word length, the price paid for answering a two-constraint query in a single lookup. Worth being able to state that trade-off explicitly if asked.

Common interview follow-ups to be ready for

- `implement_trie`: "how would you support delete?" — need to walk to the word's end node, clear `is_end`, then walk back up removing any now-childless nodes that aren't the end of some other word.
- `add_and_search_word`: "what's the worst-case time for a query full of dots?" — $O(26^{\text{len}})$ in the worst case (branching at every level); be ready to state this isn't actually $O(\text{len})$ like a plain search.
- `word_search_ii`: "why build one trie for all words instead of searching for each word separately?" — shared prefixes across words would otherwise be re-walked on the grid once per word; the shared trie prunes all of them together in one pass.
- `map_sum_pairs`: "what if `sum()` needed to support a range of prefixes, not just one?" — would need a different structure (e.g. a sorted list of keys with binary search), since a trie only naturally answers single-prefix queries efficiently.
- `short_encoding_of_words`: "can you do this without a trie?" — yes, by checking (for each word) whether it's a suffix of any other word in the list directly, but that's $O(n^2 \cdot \text{avg_len})$ versus the trie's $O(\text{total chars})$; know both and the complexity gap between them.
- `maximum_xor_of_two_numbers`: "what if numbers could be negative?" — the fixed-width two's-complement bit representation still works, but the sign bit needs special handling; be ready to discuss it even if not implementing it live.
- `prefix_and_suffix_search`: "what's the space cost, and is it worth it?" — $O(\text{len}^2)$ per word from inserting every suffix; worth contrasting with a simpler (but slower per query) approach of two separate tries intersected with a set, to show you understand the trade-off being made.

Self-check before moving on

You should be able to, without looking at the code:

- Write the basic `TrieNode/insert/search` skeleton from memory.
- Explain why `add_and_search_word`'s wildcard search needs DFS instead of a single-path walk.
- Explain the "reverse the string" trick in `short_encoding_of_words` and why it turns suffix into prefix.
- Explain how a trie generalizes to bits in `maximum_xor_of_two_numbers`, and why the greedy "opposite bit" choice at each level is safe.
- State the combined-key trick from `prefix_and_suffix_search` and what it costs in extra space.

If any of those aren't fluent yet, this topic isn't "done" per the README's definition — revisit before moving to `10_recursion_backtracking`.

Programs

01. Implement Trie

01_implement_trie.py

Explanation

Problem: Implement a trie (prefix tree) with insert(word), search(word) (exact match), and starts_with(prefix) (prefix match).

Source : LeetCode 208 - Implement Trie (Prefix Tree)

Example:

```
trie = Trie()
trie.insert("apple")
trie.search("apple") -> True
trie.search("app") -> False (only a prefix, not inserted)
trie.starts_with("app") -> True
trie.insert("app")
trie.search("app") -> True
```

Idea: the foundational data structure for this whole folder. Each node holds a dict mapping character -> child node, plus a flag marking "a complete word ends here." Shared prefixes across inserted words share the same chain of nodes automatically - "apple" and "app" share the nodes for a-p-p, and only "apple" continues past that. This is what makes a trie efficient for anything involving many strings that share prefixes: shared work is done once, not once per string. search() and starts_with() differ by exactly one thing: search() additionally requires the final node's `is_end` flag to be set (the string must be a complete inserted word, not just a path that happens to exist because it's a prefix of something longer).

Code

```
class TrieNode:
    def __init__(self):
        self.children: dict[str, "TrieNode"] = {}
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, TrieNode())
        node.is_end = True

    def _find(self, s: str):
        node = self.root
```

```

        for ch in s:
            if ch not in node.children:
                return None
            node = node.children[ch]
        return node

    def search(self, word: str) -> bool:
        node = self._find(word)
        return node is not None and node.is_end

    def starts_with(self, prefix: str) -> bool:
        return self._find(prefix) is not None

if __name__ == "__main__":
    trie = Trie()
    ops = [
        (trie.insert, ("apple",), None),
        (trie.search, ("apple",), True),
        (trie.search, ("app",), False),
        (trie.starts_with, ("app",), True),
        (trie.insert, ("app",), None),
        (trie.search, ("app",), True),
        (trie.search, ("appl",), False),
        (trie.starts_with, ("b",), False),
    ]

    for i, (fn, args, expected) in enumerate(ops, 1):
        got = fn(*args)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} ({fn.__name__}{args} -> got {got}, expected {expected})")

```

02. Add And Search Word

02_add_and_search_word.py

Explanation

Problem: Design a data structure supporting add_word(word) and search(word), where search's word may contain '.' as a wildcard matching any single character.

Source : LeetCode 211 - Design Add and Search Words Data Structure

Example:

```

wd = WordDictionary()
wd.add_word("bad"); wd.add_word("dad"); wd.add_word("mad")
wd.search("pad") -> False
wd.search("bad") -> True
wd.search(".ad") -> True
wd.search("b..") -> True

```

Idea: same trie as 01 for storage, but search can no longer be a simple single-path walk, because '.' means "try every branch here." That turns search into a DFS: at a normal character, follow the one matching child (or fail if it doesn't exist, same as before); at a '.', recursively try *every* child and succeed if any of them lead to a full match. This is exhaustive backtracking bounded by the trie's

branching factor, not a full re-scan of all stored words.

Code

```
class TrieNode:
    def __init__(self):
        self.children: dict[str, "TrieNode"] = {}
        self.is_end = False

class WordDictionary:
    def __init__(self):
        self.root = TrieNode()

    def add_word(self, word: str) -> None:
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, TrieNode())
        node.is_end = True

    def search(self, word: str) -> bool:
        def dfs(node: TrieNode, i: int) -> bool:
            if i == len(word):
                return node.is_end
            ch = word[i]
            if ch == ".":
                return any(dfs(child, i + 1) for child in node.children.values())
            if ch not in node.children:
                return False
            return dfs(node.children[ch], i + 1)

        return dfs(self.root, 0)

if __name__ == "__main__":
    wd = WordDictionary()
    ops = [
        (wd.add_word, ("bad",), None),
        (wd.add_word, ("dad",), None),
        (wd.add_word, ("mad",), None),
        (wd.search, ("pad",), False),
        (wd.search, ("bad",), True),
        (wd.search, (".ad",), True),
        (wd.search, ("b..",), True),
        (wd.search, ("....",), False),
    ]

    for i, (fn, args, expected) in enumerate(ops, 1):
        got = fn(*args)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} ({fn.__name__}{args} -> got {got}, expected {expected})")
```

03. Replace Words

03_replace_words.py

Explanation

Problem: Given a dictionary of root words and a sentence, replace every word in the sentence with its shortest matching root from the

dictionary, if one exists (a word is replaceable if some dictionary root is a prefix of it). Words with no matching root are left as-is.

Source : LeetCode 648 - Replace Words

Example:

Input: dictionary = ["cat", "bat", "rat"],

sentence = "the cattle was rattled by the battery"

Output: "the cat was rat by the bat"

Idea: build a trie from the dictionary roots, then for each word in the sentence, walk it character by character down the trie - the *first* **is_end** node hit along that walk is necessarily the shortest matching root (any root found earlier in the walk is, by definition, shorter than one found later). Stop as soon as either a root is found or the trie runs out of matching children (no root applies, keep the original word).

Code

```
class TrieNode:
    def __init__(self):
        self.children: dict[str, "TrieNode"] = {}
        self.is_end = False

def replace_words(dictionary: list[str], sentence: str) -> str:
    root = TrieNode()
    for word in dictionary:
        node = root
        for ch in word:
            node = node.children.setdefault(ch, TrieNode())
        node.is_end = True

    def find_root(word: str) -> str:
        node = root
        prefix_chars = []
        for ch in word:
            if ch not in node.children:
                break
            node = node.children[ch]
            prefix_chars.append(ch)
            if node.is_end:
                return "".join(prefix_chars)
        return word

    return " ".join(find_root(w) for w in sentence.split())

if __name__ == "__main__":
    tests = [
        (["cat", "bat", "rat"], "the cattle was rattled by the battery",
         "the cat was rat by the bat"),
        (["a", "b", "c"], "aadsfasf absbs bbab cadsfafs", "a a b c"),
        (["catt", "cat", "bat", "rat"], "the cattle was rattled by the battery",
         "the cat was rat by the bat"),
    ]

    for i, (dictionary, sentence, expected) in enumerate(tests, 1):
        got = replace_words(dictionary, sentence)
```

```
status = "PASS" if got == expected else "FAIL"
print(f"Test {i}: {status} (got {got!r}, expected {expected!r})")
```

04. Longest Word In Dictionary

04_longest_word_in_dictionary.py

Explanation

Problem: Given a list of words, find the longest word that can be built one character at a time by other words in the list (i.e. every prefix of it, at every length, is also present in the list). If there's a tie in length, return the lexicographically smallest.

Source : LeetCode 720 - Longest Word in Dictionary

Example:

Input: ["w", "wo", "wor", "worl", "world"]

Output: "world" (every prefix - w, wo, wor, worl - is present)

Idea: build a trie from all words, marking `is_end` on the node for each complete word. Then DFS from the root, but with a constraint not seen in 01-03: only descend into a child if that child's node is itself `is_end` - meaning a word ending exactly there is also in the list. This directly encodes "buildable one character at a time" as a traversal rule, rather than checking all prefixes separately for each candidate word. Visiting children in sorted character order means the first-found longest word is automatically the lexicographically smallest tie-breaker too.

Code

```
class TrieNode:
    def __init__(self):
        self.children: dict[str, "TrieNode"] = {}
        self.is_end = False

def longest_word(words: list[str]) -> str:
    root = TrieNode()
    for w in words:
        node = root
        for ch in w:
            node = node.children.setdefault(ch, TrieNode())
        node.is_end = True

    best = ""

    def dfs(node: TrieNode, path: list[str]) -> None:
        nonlocal best
        word = "".join(path)
        if len(word) > len(best) or (len(word) == len(best) and word < best):
            best = word
        for ch in sorted(node.children):
            child = node.children[ch]
            if child.is_end:
                path.append(ch)
                dfs(child, path)
```



```

        path.pop()

    dfs(root, [])
    return best

if __name__ == "__main__":
    tests = [
        (["w", "wo", "wor", "worl", "world"], "world"),
        (["a", "banana", "app", "appl", "ap", "apply", "apple"], "apple"),
        (["a"], "a"),
    ]

    for i, (words, expected) in enumerate(tests, 1):
        got = longest_word(words)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got!r}, expected {expected!r})")

```

05. Map Sum Pairs

05_map_sum_pairs.py

Explanation

Problem: Design a structure supporting insert(key, val) (overwriting val if key already exists) and sum(prefix) - the sum of vals for every inserted key that starts with prefix.

Source : LeetCode 677 - Map Sum Pairs

Example:

```

ms = MapSum()
ms.insert("apple", 3)
ms.sum("ap") -> 3
ms.insert("app", 2)
ms.sum("ap") -> 5

```

Idea: rather than storing values only at word-end nodes and summing via a DFS over the subtree at query time (correct, but $O(\text{subtree size})$ per query), store a running total *at every node along each inserted key's path*, so sum(prefix) becomes a single $O(\text{len}(\text{prefix}))$ walk to the prefix's node and a direct read. The one subtlety: overwriting an existing key must adjust every node on its path by the *difference* between the new and old value (not just add the new value again), so a separate **key_values** dict tracks each key's current value for computing that delta.

Code

```

class TrieNode:
    def __init__(self):
        self.children: dict[str, "TrieNode"] = {}
        self.value = 0

class MapSum:
    def __init__(self):
        self.root = TrieNode()

```

```

        self.key_values: dict[str, int] = {}

    def insert(self, key: str, val: int) -> None:
        delta = val - self.key_values.get(key, 0)
        self.key_values[key] = val

        node = self.root
        node.value += delta
        for ch in key:
            node = node.children.setdefault(ch, TrieNode())
            node.value += delta

    def sum(self, prefix: str) -> int:
        node = self.root
        for ch in prefix:
            if ch not in node.children:
                return 0
            node = node.children[ch]
        return node.value

if __name__ == "__main__":
    ms = MapSum()
    ops = [
        (ms.insert, ("apple", 3), None),
        (ms.sum, ("ap",), 3),
        (ms.insert, ("app", 2), None),
        (ms.sum, ("ap",), 5),
        (ms.insert, ("apple", 5), None), # overwrite: 3 -> 5
        (ms.sum, ("ap",), 7),
        (ms.sum, ("b",), 0),
    ]

    for i, (fn, args, expected) in enumerate(ops, 1):
        got = fn(*args)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} ({fn.__name__}{args} -> got {got}, expected {expected})")

```

06. Word Search II

06_word_search_ii.py

Explanation

Problem: Given an $m \times n$ grid of letters and a list of words, return all words from the list that can be formed by a path of adjacent cells (up/down/left/right, no cell reused within one word).

Source : LeetCode 212 - Word Search II

Example:

```

board = [
    ["o","a","a","n"],
    ["e","t","a","e"],
    ["i","h","k","r"],
    ["i","f","l","v"]]
words = ["oath", "pea", "eat", "rain"]
Output: ["eat", "oath"]

```

Idea: searching for each word separately with its own DFS would re-walk shared prefixes of the grid over and over (e.g. every word starting with the same letters retraces the same early cells). Instead, build one trie from *all* words first, then do a single DFS per starting cell that walks the trie and the grid *in lockstep together* - at each cell, only step to a neighbor if that neighbor's letter exists as a child in the current trie node, pruning any path that no word could possibly complete. Storing the full word at its terminal trie node (instead of just an `is_end` flag) means a match is found directly with no need to reconstruct it from the path. Temporarily marking a visited cell ('#') prevents reuse within one path, undone on backtrack so other paths can still use that cell.

Code

```
class TrieNode:
    def __init__(self):
        self.children: dict[str, "TrieNode"] = {}
        self.word: str | None = None

def find_words(board: list[list[str]], words: list[str]) -> list[str]:
    root = TrieNode()
    for w in words:
        node = root
        for ch in w:
            node = node.children.setdefault(ch, TrieNode())
        node.word = w

    rows, cols = len(board), len(board[0])
    found = set()

    def dfs(r: int, c: int, node: TrieNode) -> None:
        ch = board[r][c]
        if ch not in node.children:
            return
        next_node = node.children[ch]
        if next_node.word:
            found.add(next_node.word)

        board[r][c] = "#"
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and board[nr][nc] != "#":
                dfs(nr, nc, next_node)
        board[r][c] = ch

    for r in range(rows):
        for c in range(cols):
            dfs(r, c, root)

    return list(found)

if __name__ == "__main__":
    tests = [
        (["o", "a", "a", "n"], ["e", "t", "a", "e"], ["i", "h", "k", "r"], ["i", "f", "l", "v"]],
        ["oath", "pea", "eat", "rain"], {"eat", "oath"}),
        (["a", "b"], ["c", "d"], ["abcb"], set()),
    ]
```

```

for i, (board, words, expected) in enumerate(tests, 1):
    got = set(find_words(board, words))
    status = "PASS" if got == expected else "FAIL"
    print(f"Test {i}: {status} (got {got}, expected {expected})")

```

07. Search Suggestions System

07_search_suggestions_system.py

Explanation

Problem: Given a list of products and a search word, for every prefix of search_word (built one character at a time as if the user were typing), return up to 3 lexicographically smallest products that have that prefix.

Source : LeetCode 1268 - Search Suggestions System

Example:

products = ["mobile", "mouse", "moneypot", "monitor", "mousepad"]

search_word = "mouse"

Output: [

["mobile", "moneypot", "monitor"], # prefix "m"

["mobile", "moneypot", "monitor"], # prefix "mo"

["mouse", "mousepad"], # prefix "mou"

["mouse", "mousepad"], # prefix "mous"

["mouse", "mousepad"], # prefix "mouse"

]

Idea: the "autocomplete" trie pattern - sort all products first, then insert them into a trie in that sorted order, capping each node's stored suggestion list at 3 entries. Because insertion happens in sorted order and each node only *accepts* new entries while its list has fewer than 3, every node's list ends up holding exactly the (up to) 3 lexicographically smallest products passing through it - no separate sort needed at query time. Answering the query is then just walking the trie one character at a time along search_word and reading off each node's precomputed list directly.

Code

```

class TrieNode:
    def __init__(self):
        self.children: dict[str, "TrieNode"] = {}
        self.suggestions: list[str] = []

def suggested_products(products: list[str], search_word: str) -> list[list[str]]:
    root = TrieNode()
    for p in sorted(products):
        node = root
        for ch in p:
            node = node.children.setdefault(ch, TrieNode())
            if len(node.suggestions) < 3:
                node.suggestions.append(p)

```

```

result = []
node = root
for ch in search_word:
    if node is not None and ch in node.children:
        node = node.children[ch]
        result.append(node.suggestions)
    else:
        node = None
        result.append([])

return result

if __name__ == "__main__":
    tests = [
        (["mobile", "mouse", "moneypot", "monitor", "mousepad"], "mouse", [
            ["mobile", "moneypot", "monitor"],
            ["mobile", "moneypot", "monitor"],
            ["mouse", "mousepad"],
            ["mouse", "mousepad"],
            ["mouse", "mousepad"],
        ]),
        (["havana", "havana", ["havana"] * 6],
        (["bags", "baggage", "banner", "box", "cloths"], "bags", [
            ["baggage", "bags", "banner"],
            ["baggage", "bags", "banner"],
            ["baggage", "bags"],
            ["bags"],
        ]),
    ]

    for i, (products, word, expected) in enumerate(tests, 1):
        got = suggested_products(products, word)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```

08. Short Encoding Of Words

08_short_encoding_of_words.py

Explanation

Problem: A valid encoding of a list of words is a reference string built by concatenating each word followed by '#', where every word in the list appears as a *suffix* of some entry in the reference (so a word doesn't need its own entry if it's already a suffix of a longer word already included). Return the length of the shortest possible valid encoding.

Source : LeetCode 820 - Short Encoding of Words

Example:

Input: ["time", "me", "bell"]

Output: 10 ("time#bell#" - "me" is a suffix of "time", so it doesn't need its own entry)

Idea: "is a suffix of" on normal words becomes "is a *prefix* of" if every word is reversed first - and prefix relationships are exactly what a trie represents naturally (shared prefixes share nodes). Build

a trie from all **reversed** words. A word only needs its own encoding entry if no other word extends past it in the trie - i.e. its reversed-word node has no children (a leaf). Any word whose node **does** have children is a suffix of some longer word already covered by that longer word's own entry, so it contributes nothing extra.

Code

```
class TrieNode:
    def __init__(self):
        self.children: dict[str, "TrieNode"] = {}

def minimum_length_encoding(words: list[str]) -> int:
    words = list(set(words)) # exact duplicates never need re-encoding
    root = TrieNode()
    node_for_word = {}

    for w in words:
        node = root
        for ch in reversed(w):
            node = node.children.setdefault(ch, TrieNode())
        node_for_word[w] = node

    total = 0
    for w in words:
        if not node_for_word[w].children: # leaf: not a suffix of any other word
            total += len(w) + 1

    return total

if __name__ == "__main__":
    tests = [
        (["time", "me", "bell"], 10),
        (["t"], 2),
        (["time", "me"], 5),
        (["a", "aa", "aaa"], 4),
    ]

    for i, (words, expected) in enumerate(tests, 1):
        got = minimum_length_encoding(words)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

09. Maximum Xor Of Two Numbers

09_maximum_xor_of_two_numbers.py

Explanation

Problem: Given an integer array `nums`, return the maximum possible XOR of any two numbers in it.

Source : LeetCode 421 - Maximum XOR of Two Numbers in an Array

Example:

Input: [3, 10, 5, 25, 2, 8]

Output: 28 (5 XOR 25 = 28)

Idea: a trie doesn't have to store characters - it can store the *bits* of a number, one level per bit, from most significant to least. Every number becomes a fixed-length root-to-leaf path of 0s and 1s. Insert every number's bit-path into the trie first. Then for each number, greedily walk the trie trying, at every bit, to go toward the *opposite* bit of the current number - XOR-ing opposite bits produces a 1 in that position, maximizing the result from the most significant bit downward (the greedy choice is safe because a 1 in a higher bit position always beats any combination of lower bits). If the opposite branch doesn't exist, fall back to the same bit (contributing a 0 there) and keep going. This finds, for every number, its best XOR partner in $O(\text{bit_length})$ instead of comparing all $O(n^2)$ pairs.

Code

```
class TrieNode:
    def __init__(self):
        self.children: dict[int, "TrieNode"] = {}

def find_maximum_xor(nums: list[int]) -> int:
    if not nums:
        return 0

    max_bits = max(nums).bit_length()
    root = TrieNode()

    for n in nums:
        node = root
        for i in range(max_bits - 1, -1, -1):
            bit = (n >> i) & 1
            node = node.children.setdefault(bit, TrieNode())

    best = 0
    for n in nums:
        node = root
        curr_xor = 0
        for i in range(max_bits - 1, -1, -1):
            bit = (n >> i) & 1
            toggled = 1 - bit
            if toggled in node.children:
                curr_xor = (curr_xor << 1) | 1
                node = node.children[toggled]
            else:
                curr_xor = curr_xor << 1
                node = node.children[bit]
        best = max(best, curr_xor)

    return best

if __name__ == "__main__":
    tests = [
        ([3, 10, 5, 25, 2, 8], 28),
        ([0], 0),
        ([2, 4], 6),
        ([8, 10, 2], 10),
    ]
```

```

for i, (inp, expected) in enumerate(tests, 1):
    got = find_maximum_xor(inp)
    status = "PASS" if got == expected else "FAIL"
    print(f"Test {i}: {status} (got {got}, expected {expected})")

```

10. Prefix And Suffix Search

10_prefix_and_suffix_search.py

Explanation

Problem: Design WordFilter(words), supporting f(prefix, suffix):
 return the largest index i such that words[i] has the given prefix
 AND the given suffix (both simultaneously). Return -1 if none match.
 Source : LeetCode 745 - Prefix and Suffix Search

Example:

```

wf = WordFilter(["apple", "app"])
wf.f("a", "e") -> 0 ("apple" starts with 'a', ends with 'e')
wf.f("a", "p") -> 1 ("app" starts with 'a', ends with 'p')

```

Idea: 01-09 all trie on a *single* dimension per word (its prefix structure alone). This problem needs "starts with X AND ends with Y" simultaneously, which a plain prefix trie can't answer directly. The trick: for each word, insert every one of its (suffix, word) combinations as a *single combined key*: `suffix + "#" + word`, for every possible suffix length. A query for (prefix, suffix) then becomes a single ordinary trie lookup of `suffix + "#" + prefix` - if that combined path exists in the trie, every word that passes through it necessarily has both the required suffix (it's literally the start of the key) and the required prefix (since the word itself, appearing right after the '#', starts with it). Storing the word's index at every node along each inserted path, overwritten in increasing index order, means the last (largest) index naturally survives at each node - exactly the tie-breaking rule the problem asks for.

Code

```

class TrieNode:
    def __init__(self):
        self.children: dict[str, "TrieNode"] = {}
        self.index = -1

class WordFilter:
    def __init__(self, words: list[str]):
        self.root = TrieNode()
        for idx, word in enumerate(words):
            for j in range(len(word) + 1):
                key = word[j:] + "#" + word
                node = self.root
                node.index = idx
                for ch in key:
                    node = node.children.setdefault(ch, TrieNode())

```



```

        node.index = idx

def f(self, prefix: str, suffix: str) -> int:
    key = suffix + "#" + prefix
    node = self.root
    for ch in key:
        if ch not in node.children:
            return -1
        node = node.children[ch]
    return node.index

if __name__ == "__main__":
    wf = WordFilter(["apple", "app"])
    tests = [
        ("a", "e"), 0),
        ("a", "p"), 1),
        ("b", "e"), -1),
        ("app", "p"), 1),
    ]

    for i, ((prefix, suffix), expected) in enumerate(tests, 1):
        got = wf.f(prefix, suffix)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (f({prefix!r}, {suffix!r}) -> got {got}, expected {expected})")

```